

Structured Agent — Language Overview

What it is

An experimental language that interleaves LLM calls with deterministic procedure calls.

- The programmer defines the process
- The LLM fills gaps — parameters, typed responses, decisions
- Context is managed per call scope — not by the programmer

Function Syntax

```
fn analyze(code: String): Analysis {  
    "Analyze this code"!  
    code!  
}
```

```
fn main(): () {  
    let result = analyze(code)  
}
```

- No ctx: Context parameter — context is implicit, scoped to the call
- Return type uses :
- Functions with prompt injections return LLM-generated values

Implicit Context

```
fn analyze(code: String): Analysis {  
    "You are a code reviewer!"  
    code!  
}
```

```
fn main(): () {  
    "You are a senior engineer!"  
    let result = analyze(code)  
    // analyze's context dropped here  
}
```

LLM call 1 sees:

You are a senior engineer
You are a code reviewer
<code>

After return:

You are a senior engineer

Structs

```
struct Address {  
    city: String,  
}  
  
struct Person {  
    name: String,  
    address: Address,  
}  
  
fn main(): String {  
    let p = Person {  
        name: "Alice",  
        address: Address { city: "London" }  
    }  
    return p.address.city  
}
```

- Declared with struct
- Nested structs
- Field access with .
- Can be injected: r!
- Can be LLM return types

Generics

```
struct Wrapper<T> {  
    label: String,  
    value: T,  
}
```

```
struct Profile {  
    name: String,  
    bio: Option<String>,  
}
```

```
fn main(): List<Task> {  
    return [  
        Task { title: "write tests", done:  
↪ true },  
        Task { title: "fix bugs",    done:  
↪ false }  
    ]  
}
```

- `struct Name<T>` — generic structs
- `Option<T>` — nullable fields
- `List<T>` — typed list literals
- Type parameters guide the LLM's JSON response schema

Module System

greeter.sa

```
mod greeter
```

```
pub fn greet(name: String): String {  
    return name  
}
```

main.sa

```
use greeter::greet
```

```
fn main(): () {  
    let result = greet("Alice")  
    result!  
}
```

- `mod name` declares the module
- `pub fn` exports a function
- `use module::fn` imports it
- Files discovered from the working directory

Module Interfaces: sig

formatter.sa

```
mod formatter
```

```
sig Formatter {  
    fn format(value: String): String  
}
```

```
pub fn format(value: String): String {  
    return value  
}
```

reporter.sa

```
mod reporter(fmt: formatter::Formatter)
```

```
pub fn report(value: String): () {  
    let formatted = fmt::format(value)  
    formatted!  
}
```

main.sa

```
mod reporter(formatter)
```

```
use reporter::report
```

```
fn main(): () {  
    report("hello")  
}
```

- sig declares a structural interface
- Modules can take other modules as parameters
- Dispatch is vtable-based at runtime
- Dependency injection without an object system

Actors: Declaration

```
struct State {}

mod Worker(state: State) {
  pub fn run(): () {
    "Starting work"!
    while true {
      "Next step"!
      yield
    }
  }

  pub fn status(): String {
    "Current status"!
  }
}
```

- `mod Name(state)` — actor module with state
- `pub fn` — methods callable from outside
- `yield` — suspends the actor, lets other work proceed
- Each actor has its own LLM context

Actors: Spawning and Calling

```
fn main(): String {  
    let w = spawn<Worker>("w1", State{})  
    defer w::run()  
    let s = w::status()  
    return s  
}
```

- `spawn<Name>("id", state)` — creates an instance
- `w::fn()` — calls a method on the actor
- `defer w::fn()` — starts running concurrently, suspends at `yield`
- `w::status()` runs to completion while `run` is suspended

Select Statement

```
let result = select {  
  add(_, _),  
  subtract(_, _)  
}
```

- LLM sees the accumulated context and the list of functions
- Picks one branch and fills any _ parameters
- The result of the chosen branch is the value of the expression

Select: Step by Step

```
fn agent_loop(): String {  
    "what is the next action?"!  
  
    let result = select {  
        read_file(_),  
        write_file(_, _),  
        list_directory(_)  
    }  
  
    result!  
  
    if is_task_complete() {  
        return summarize()  
    } else {  
        return agent_loop()  
    }  
}
```

For: “add SUCCESS to test.tmp”

1. LLM selects `read_file(_)`, fills "test.tmp"
2. Contents returned, injected via `result!`
3. `is_task_complete()` → false
4. Recurse — context now includes file contents
5. LLM selects `write_file(_, _)`, fills path + new content
6. `is_task_complete()` → true → `summarize()`

Standard Library

```
use io::print
use io::input
use messaging::receive
use unstable::head
```

Module	Functions
io	print, input
messaging	receive
fs	file system ops
math	arithmetic
logic	boolean helpers
equality	equality checks
unstable	head, tail, option helpers
actor	actor system support

LLM Backends

```
# Gemini
sa run --engine gemini
      --gemini-api-key KEY
      --gemini-model gemini-2.5-pro

# OpenAI
sa run --engine openai
      --openai-api-key KEY
      --openai-model gpt-4o

# Hugging Face (OpenAI-compatible)
sa run --engine huggingface
      --hf-token TOKEN
      --hf-model Qwen/Qwen2.5-72B-Instruct
```

- All three backends implement the same `LanguageEngine` trait
- Same `.sa` program runs unchanged against any backend
- HuggingFace reuses the OpenAI engine with a different base URL
- Config can live in `config.toml` instead of CLI flags

LLM Thinking (Gemini)

Level	Budget
disabled	0 tokens
low	512 tokens
medium	model-determined
high	model-determined
with_budget(n)	explicit

- Configured on the engine before the runtime starts
- The model reasons internally before generating each response
- Thoughts are stored as `ThinkingEvent` in context
- Fed back to the model on subsequent turns alongside regular context

A full agent

Agent: Structure

```
fn main(): () {  
    '''  
    You are a structured agent.  
    Use print to respond to users.  
    '''!  
  
    let history = read_file("history.md")  
  
    while true {  
        history!  
        let task = receive()  
        task!  
        plan()!  
        let summary = agent_loop()  
        history = update_history(summary)  
        write_file("history.md", history)  
    }  
}
```

```
fn agent_loop(): String {  
    "what is the next action?"!  
  
    let result = select {  
        plan(),  
        edit_file(),  
        read_file(),  
        write_file_content(_, _),  
        list_directory(),  
        grep(_, _, _, _, _),  
        execute_command(),  
        ask_for_more_info()  
    }  
  
    result!  
  
    if is_task_complete() {  
        return summarize()  
    } else {  
        return agent_loop()  
    }  
}
```

Agent: What the Structure Enforces

- The outer loop **always** reads history, receives a task, plans, loops, then writes history
- The inner loop **always** checks `is_task_complete()` before stopping
- The model **cannot** skip planning, skip writing history, or exit early
- Prompt injection through tool results passes through `check_content()` — enforced by the language, not the model

Summary

Language features added since v1:

- Implicit context (no ctx param)
- Structs with generics
- `Option<T>`, `List<T>`
- Full module system
- sig interfaces
- Actors with `yield` and `defer`
- Simplified `select`
- Standard library
- Multi-line `'''...'''` strings

Runtime features:

- Gemini, OpenAI, Hugging Face backends
- LLM thinking support
- Struct schema inference for typed LLM responses
- Actor registry and scheduler

The key question remains: how small can the model be before structured execution makes up the difference?